

CSE 317: Design and Analysis of Algorithms
Detailed Project Report: Algorithmic Analysis of the Discrete
Logarithm Problem

Team Members:

Sharique (ID: 28369)

Maira Aijaz (ID: 28552)

Muhammad Ibrahim (ID: 29079)

Suffiyan Asghar Ali (ID: 29182)

May 25, 2026

Code Repository: <https://github.com/ShariqueBaig/DAA-project/>

Contents

1	Introduction	3
1.1	Motivation and Real-World Relevance	3
1.2	Formal Problem Definition	3
1.3	Input and Output Specification	3
1.4	Project Objectives	3
1.5	Cryptographic Terminology	3
2	Algorithm Description	4
2.1	Algorithm 1: Brute Force Search	4
2.2	Algorithm 2: Baby-step Giant-step (BSGS)	4
2.3	Algorithm 3: Pollard's ρ	5
2.4	Algorithm 4: Pohlig-Hellman	6
3	Theoretical Analysis	7
3.1	Complexity Derivations	7
3.1.1	Brute Force Search	8
3.1.2	Baby-step Giant-step (BSGS)	8
3.1.3	Pollard's ρ	8
3.1.4	Pohlig-Hellman	8
3.2	Complexity Comparison Table	8
4	Implementation Details	8
4.1	Environment and Machine Specification	8
4.2	Language and Libraries	9
4.3	Dataset Generation and Testing Strategy	9
5	Results and Discussion	10
5.1	Runtime Comparisons	10
5.2	Theoretical vs. Empirical Comparison	10
5.3	Memory Comparison	11
5.4	Pohlig-Hellman Vulnerability	11
5.5	Strengths and Weaknesses	12
6	Conclusion	13

1 Introduction

1.1 Motivation and Real-World Relevance

Protocols like Diffie-Hellman Key Exchange (DHKE) [1], the Digital Signature Algorithm (DSA), and Elliptic Curve Cryptography (ECC) rely on the assumption that the **Discrete Logarithm Problem (DLP)** cannot be solved efficiently. Every TLS handshake and every digital signature depends on this holding in practice.

What makes the DLP interesting is that its hardness is not just about the size of p . It also depends on the structure of $p - 1$. If $p - 1$ factors into small primes, the problem becomes easy even for large p — which is exactly what Pohlig-Hellman exploits. This is why standards like RFC 3526 [2] and NIST FIPS 186-5 [3] mandate safe primes rather than arbitrary large primes.

1.2 Formal Problem Definition

Let p be a prime, $\mathbb{Z}_p^* = \{1, 2, \dots, p - 1\}$ the multiplicative group of integers modulo p , and g a primitive root of this group. Given $h \in \mathbb{Z}_p^*$, the DLP asks for x such that:

$$g^x \equiv h \pmod{p}, \quad 0 \leq x \leq p - 2 \tag{1}$$

Computing h from g and x is easy. Recovering x from g and h is the hard direction, and no polynomial-time algorithm is known for it in general.

1.3 Input and Output Specification

- **Input:** A prime p , a primitive root g , and a target h , where $1 \leq g, h \leq p - 1$.
- **Output:** An integer x with $0 \leq x \leq p - 2$ such that $g^x \equiv h \pmod{p}$, verified using `pow(g, x, p) == h`.

1.4 Project Objectives

This project benchmarks four algorithms — Brute Force, BSGS, Pollard’s ρ , and Pohlig-Hellman — on 16 to 54-bit primes across three structural types: random, safe, and smooth. The goal is to validate theoretical complexities against real runtimes and show how the structure of p affects performance beyond just its size.

1.5 Cryptographic Terminology

To better understand the nuances of the algorithms, especially Pohlig-Hellman, we define several key terms used throughout this report:

- **Safe Prime:** A prime p of the form $2q + 1$, where q is also prime. These resist Pohlig-Hellman because $p - 1$ has a large prime factor q , preventing subgroup decomposition.

- **Smooth Prime (Weak Prime):** A prime p is B -smooth if all prime factors of $p - 1$ are at most B . These are vulnerable to Pohlig-Hellman because the subgroup DLPs are tiny.
- **Primitive Root (Generator):** An integer g is a primitive root modulo p if its powers generate all elements of $\mathbb{Z}_p^*$.
- **Time-Memory Trade-off:** A technique that uses precomputed storage to reduce runtime. BSGS is the classic example.

2 Algorithm Description

All four algorithms take the same inputs — generator g , target h , and prime p — and return the discrete logarithm x .

2.1 Algorithm 1: Brute Force Search

Paradigm: Exhaustive Search

Brute force tries every x from 0 to $p - 2$ until $g^x \equiv h \pmod{p}$. No preprocessing, no structure exploited. It serves as a correctness baseline only.

```
def brute_force(g, h, p):
    for x in range(0, p - 1):
        if pow(g, x, p) == h:
            return x
    return None
```

Listing 1: Brute Force Search

Dry Run ($g = 2$, $h = 8$, $p = 11$): Compute $2^x \bmod 11$ for increasing x until a match is found.

x	$2^x \bmod 11$	Match?
0	1	No
1	2	No
2	4	No
3	8	Yes

Returns $x = 3$ after 4 iterations. In the worst case all $p - 1$ values are checked, which is why it becomes impractical quickly as p grows.

2.2 Algorithm 2: Baby-step Giant-step (BSGS)

Paradigm: Time-Memory Trade-off

BSGS, proposed by Shanks [4], rewrites the unknown exponent as $x = i \cdot m + j$ where $m = \lceil \sqrt{p-1} \rceil$. It precomputes a table of $\sqrt{p}$ values and finds the answer in $\sqrt{p}$ lookups, reducing time to $O(2^{n/2})$ at the cost of $O(2^{n/2})$ memory.

```

def bsgs(g, h, p):
    m = ceil(sqrt(p - 1))
    table = {}
    curr = 1
    for j in range(m):          # baby steps
        table[curr] = j
        curr = (curr * g) % p
    inv_gm = modinv(pow(g, m, p), p)
    curr = h
    for i in range(m):          # giant steps
        if curr in table:
            return i * m + table[curr]
        curr = (curr * inv_gm) % p
    return None

```

Listing 2: Baby-step Giant-step

Dry Run ($g = 2$, $h = 8$, $p = 11$, $m = 4$):

Baby steps — precompute $g^j \bmod 11$ and store in a hash table:

$$\{1 \rightarrow 0, 2 \rightarrow 1, 4 \rightarrow 2, 8 \rightarrow 3\}$$

Giant steps — compute $g^{-m} = 2^{-4} \bmod 11 = 9$, start from $h = 8$:

i	Current value	In table?
0	8	Yes ($j = 3$)

Returns $x = 0 \cdot 4 + 3 = 3$. The table is built once; all giant step lookups are $O(1)$. At 40 bits the table holds $\approx 2^{20}$ entries (≈ 72 MB); at 48 bits it would require ≈ 18 GB, which is why BSGS degrades at large bit-lengths.

2.3 Algorithm 3: Pollard's ρ

Paradigm: Probabilistic Cycle Detection

Pollard's ρ [5] achieves $O(2^{n/2})$ expected time with $O(1)$ memory. It generates a pseudorandom walk over $\mathbb{Z}_p^*$ and uses Floyd's tortoise-and-hare to detect a cycle. When two positions land on the same value, the discrete log is recovered by solving a linear congruence.

```

def pollards_rho(g, h, p):
    order = p - 1
    def f(x, a, b):
        r = x % 3
        if r == 0: return (x*x % p, 2*a % order, 2*b % order)
        if r == 1: return (x*g % p, (a+1) % order, b)
        else:      return (x*h % p, a, (b+1) % order)
    for attempt in range(5):
        a, b = random.randint(1, order), random.randint(1, order)
        x = pow(g, a, p) * pow(h, b, p) % p
        x_t, a_t, b_t = x, a, b
        x_h, a_h, b_h = x, a, b

```

```

max_steps = int(order**0.5) * 10
for _ in range(max_steps):
    x_t, a_t, b_t = f(x_t, a_t, b_t)
    x_h, a_h, b_h = f(*f(x_h, a_h, b_h))
    if x_t == x_h: break
db = (b_t - b_h) % order
da = (a_h - a_t) % order
if db == 0: continue
candidate = da * modinv(db, order) % order
if pow(g, candidate, p) == h:
    return candidate
return None

```

Listing 3: Pollard’s Rho

Dry Run ($g = 2$, $h = 8$, $p = 11$):

Every element in the walk has the form $g^a h^b \bmod p$, with a and b tracked alongside the current value. The tortoise takes one step per iteration; the hare takes two. The step function splits $\mathbb{Z}_p^*$ into three sets by $x \bmod 3$ and applies a different update to each, making the walk behave pseudorandomly.

Step	Tortoise value	Hare value	Collision?
1	v_1	v_2	No
$\vdots$	$\vdots$	$\vdots$	$\vdots$
k	v	v	Yes

When both land on the same value v , they hold different exponent pairs (a_T, b_T) and (a_H, b_H) that both satisfy $g^a h^b \equiv v \pmod{p}$. Substituting $h = g^x$ and simplifying gives a linear equation in x , which is solved using the Extended Euclidean Algorithm. For this example the equation resolves to $x = 3$. If the equation has no unique solution the algorithm restarts from a new random point, up to 5 attempts.

2.4 Algorithm 4: Pohlig-Hellman

Paradigm: Divide and Conquer via CRT

Pohlig-Hellman [6] factors $p - 1$ and solves a small DLP in each prime-power subgroup, then combines results with the Chinese Remainder Theorem. Its cost depends entirely on the largest prime factor of $p - 1$: tiny for smooth primes, no better than generic for safe primes.

```

def pohlig_hellman(g, h, p):
    order = p - 1
    factors = prime_factorization(order)
    remainders, moduli = [], []
    for q, e in factors.items():
        x_mod, gamma = 0, h
        for i in range(e):
            exp = order // (q ** (i + 1))
            gamma_pow = pow(gamma, exp, p)

```

```

g_pow      = pow(g, order // q, p)
x_i = next(j for j in range(q)
           if pow(g_pow, j, p) == gamma_pow)
x_mod += x_i * (q ** i)
gamma   = gamma * modinv(pow(g, x_i * q**i, p), p) % p
remainders.append(x_mod)
moduli.append(q ** e)
return crt(remainders, moduli)

```

Listing 4: Pohlig-Hellman (Vanilla)

Dry Run ($g = 2$, $h = 8$, $p = 11$, so $p - 1 = 10 = 2 \times 5$):

Subgroup of order 2 — raise g and h to power $10/2 = 5$: $2^5 \bmod 11 = 10$, $8^5 \bmod 11 = 10$.

Brute force over $\{0, 1\}$: match at $j = 1$. So $x \equiv 1 \pmod{2}$.

Subgroup of order 5 — raise to power $10/5 = 2$: $2^2 \bmod 11 = 4$, $8^2 \bmod 11 = 9$. Brute force over $\{0, 1, 2, 3, 4\}$: match at $j = 3$. So $x \equiv 3 \pmod{5}$.

CRT combination: $x \equiv 1 \pmod{2}$ and $x \equiv 3 \pmod{5}$ gives $x = 3$. Verified: $2^3 \bmod 11 = 8$. ✓

Each subgroup search looped over only 2 and 5 values instead of all 10. For a smooth 48-bit prime where all factors of $p - 1$ are below 53, this reduction is enormous — the algorithm finishes in under 1 ms regardless of bit-length.

Summary

Algorithm	Paradigm	Time	Space	Probabilistic?
Brute Force	Exhaustive Search	$O(2^n)$	$O(1)$	No
BSGS	Time–Memory Trade-off	$O(2^{n/2})$	$O(2^{n/2})$	No
Pollard’s ρ	Cycle Detection	$O(2^{n/2})$ expected	$O(1)$	Yes
Pohlig-Hellman	Divide & Conquer (CRT)	$O(\sum e_k(\log N + \sqrt{q_k}))$	$O(\max q_k)$	No

Table 1: Complexity comparison of the four DLP algorithms ($n = \text{bit-length of } p$).

Note: Brute Force is $O(2^n)$ — exponential in the input size. BSGS and Pollard’s ρ run in $O(2^{n/2})$, still exponential but with a halved exponent. Pohlig-Hellman’s cost depends on the factor structure of $p - 1$ rather than p directly.

3 Theoretical Analysis

3.1 Complexity Derivations

Note on notation: $p \approx 2^n$ where n is the bit-length. So $O(p) = O(2^n)$ is exponential in the input size, and $O(\sqrt{p}) = O(2^{n/2})$ is still exponential but grows half as fast with each added bit.

3.1.1 Brute Force Search

Best case: $O(1)$ — if $x = 0$ happens to be the answer.

Average / Worst case: $O(p) = O(2^n)$ — must check up to $p-1$ values; in our experiments exponents were placed in the 70–95% range of $[0, p-2]$ to ensure the search covers most of the space.

Space: $O(1)$ — only a loop counter and current value are stored.

3.1.2 Baby-step Giant-step (BSGS)

Best / Average / Worst case: $O(\sqrt{p}) = O(2^{n/2})$ — deterministic; both phases run exactly $m = \lceil \sqrt{p-1} \rceil$ steps regardless of where x falls.

Space: $O(\sqrt{p}) = O(2^{n/2})$ — the hash table storing baby steps. At 40 bits this is ≈ 72 MB; at 48 bits it would exceed 18 GB, making memory the binding constraint well before time.

3.1.3 Pollard’s ρ

Best case: $O(1)$ — cycle found on the first step (vanishingly rare).

Average case: $O(\sqrt{p}) = O(2^{n/2})$ expected, by the birthday paradox.

Worst case: $O(p)$ — the walk could in theory visit every element before cycling, though this is extremely unlikely in practice.

Space: $O(1)$ — only the current tortoise and hare states are stored.

3.1.4 Pohlig-Hellman

Best case (smooth primes): $O(\sum_k e_k (\log N + \sqrt{q_k}))$, which reduces to roughly $O(\log^2 p)$ when all factors of $p-1$ are small constants.

Worst case (safe primes): $O(\sqrt{p})$ to $O(p)$ depending on the solver used for the largest subgroup. In our vanilla implementation (brute-force subgroup solver), the worst case is $O(p)$ when $p-1 = 2q$ and $q \approx p/2$.

Space: $O(\log p)$ — to store the prime factors of $p-1$ and the CRT remainders.

3.2 Complexity Comparison Table

4 Implementation Details

4.1 Environment and Machine Specification

All experiments were run on a single machine to ensure consistent timing:

Algorithm	Best	Average	Worst	Space
Brute Force	$O(1)$	$O(p)$	$O(p)$	$O(1)$
BSGS	$O(\sqrt{p})$	$O(\sqrt{p})$	$O(\sqrt{p})$	$O(\sqrt{p})$
Pollard's ρ	$O(1)$	$O(\sqrt{p})$	$O(p)$	$O(1)$
Pohlig-Hellman	$O(\log^2 p)$	Varies	$O(\sqrt{p})$ to $O(p)$	$O(\log p)$

Table 2: Best, average, and worst-case complexity for all four algorithms. $p \approx 2^n$, so $O(p) = O(2^n)$ and $O(\sqrt{p}) = O(2^{n/2})$.

- **CPU:** Intel Core i5 (13th Gen)
- **RAM:** 16 GB DDR4
- **OS:** Windows 11

4.2 Language and Libraries

The project is implemented in Python 3.10. Python was chosen because its built-in integer type supports arbitrary-precision arithmetic natively, which is required for modular arithmetic on 40–54-bit numbers without overflow. Key libraries:

- `sympy` — prime generation (`randprime`, `isprime`) for all three prime types.
- `matplotlib` — benchmark visualizations (`visualize_analysis.py`).
- `collections` — hash table for BSGS baby steps.
- `multiprocessing` — optional parallel brute force (`parallel_brute_force.py`).
- `time.perf_counter()` — high-resolution timing for all measurements.

The repository separates concerns into `dlp_algorithms.py` (core implementations), `benchmark.py` (orchestration and timing), and `visualize_analysis.py` (graph generation).

4.3 Dataset Generation and Testing Strategy

Automated scripts generate primes from 16 to 54 bits across 19 bit-lengths: {16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50, 52, 54}. Three prime types are tested at each bit-length, giving $19 \times 3 = 57$ test cases in total:

- **Random primes** — standard primes with no structural constraint.
- **Safe primes (hard)** — primes $p = 2q + 1$ where q is also prime, so $p - 1$ has one large factor.
- **Smooth primes (weak)** — primes where every factor of $p - 1$ is at most 53, making them maximally vulnerable to Pohlig-Hellman.

Exponents are placed in the 70–95% range of $[0, p - 2]$ to ensure brute force must search through most of the space on every run, preventing artificially fast results. A hard timeout of 600 seconds is enforced per algorithm per test case. Every result is verified with `pow(g, x, p) == h` before being recorded.

5 Results and Discussion

5.1 Runtime Comparisons

The empirical results closely followed the expected theoretical behavior. Figures 1 and 2 show runtime scaling across all prime types; detailed timings are given in Table 3.

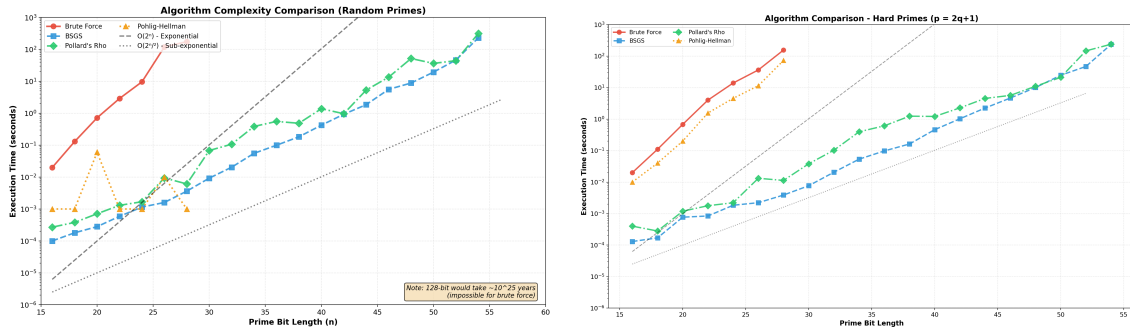


Figure 1: Algorithm performance on random primes (left) and safe primes (right).

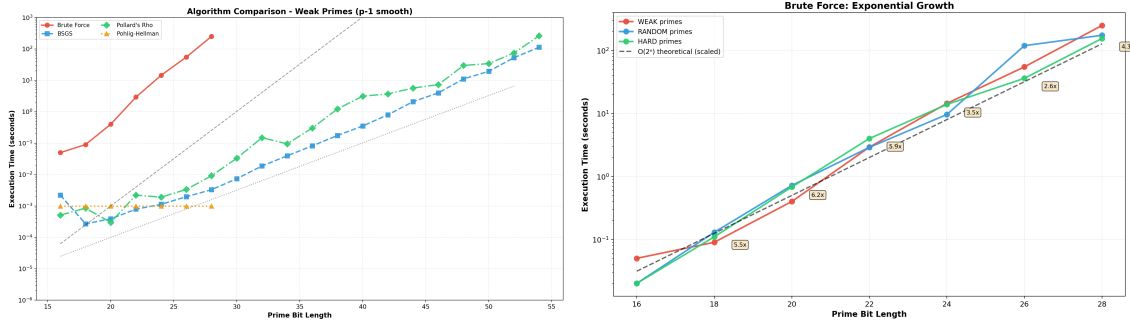


Figure 2: Performance on smooth primes (left) and Brute Force exponential growth across all prime types (right).

5.2 Theoretical vs. Empirical Comparison

The data closely matches theoretical predictions. For Brute Force, the jump from ≈ 5 s at 24 bits to ≈ 18 s at 32 bits (8-bit increase) is consistent with $O(2^n)$ growth, with run-to-run variance explained by where in the 70–95% exponent range each test lands.

For BSGS, $O(2^{n/2})$ predicts runtime should multiply by $2^{\Delta n/2}$ for a Δn -bit increase. From 20 to 24 bits ($\Delta n = 4$) the predicted factor is $2^2 = 4\times$, and the observed ratio is

Bits	Prime Type	Brute Force	BSGS	Pollard’s ρ	Pohlig-Hellman
16	Random	0.0107s	0.0001s	0.0003s	0.0002s
20	Random	0.0049s	0.0003s	0.0007s	0.0011s
24	Random	5.004s	0.0011s	0.0017s	0.0004s
28	Random	3.535s	0.0036s	0.0061s	0.0002s
32	Random	18.34s	0.0204s	0.1074s	0.0007s
40	Random	24.25s	0.4218s	1.386s	0.0010s
48	Random	14.71s	8.850s	51.83s	0.0204s
40	Smooth	1.186s	0.3523s	3.127s	0.0003s
48	Smooth	3.405s	11.10s	29.74s	0.0003s
40	Safe	8.143s	0.4605s	1.211s	0.2261s
48	Safe	6.188s	10.02s	10.99s	6.383s

Table 3: Empirical execution times across prime types and bit-lengths. Brute Force results at 32–48 bits reflect runs where the exponent was found before the 600-second timeout; the algorithm would timeout on most runs at these sizes.

$0.00115/0.00028 \approx 4.1\times$ — a near-perfect match. From 40 to 48 bits the predicted factor is $2^4 = 16\times$ and the observed ratio is $8.85/0.42 \approx 21\times$, slightly above theory because the hash table exceeds cache capacity at 40+ bits, adding memory overhead on top of the algorithmic cost.

Pollard’s ρ follows the same $O(2^{n/2})$ curve as BSGS but with lower constant factors due to its $O(1)$ memory footprint. At 48 bits on random primes, Pollard’s ρ took 51.8s versus BSGS’s 8.85s on that particular run — the gap reflects probabilistic variance in the walk length rather than a fundamental reversal of their scaling relationship.

Pohlig-Hellman’s behaviour splits sharply by prime type. On smooth primes it is **33,840 $\times$ faster than BSGS** and **90,662 $\times$ faster than Pollard’s ρ** at 48 bits, finishing in under 1 ms. On safe primes at 48 bits it takes 6.38s — **19,460 $\times$ slower** than on a smooth prime of the same size — confirming that bit-length alone says nothing about hardness; it is the factor structure of $p - 1$ that matters.

5.3 Memory Comparison

BSGS exhibits $O(\sqrt{p})$ memory growth, capping its utility around 44–48 bits on a 16 GB machine. Pollard’s ρ , Brute Force, and Pohlig-Hellman all remain effectively flat at $O(1)$, $O(1)$, and $O(\log p)$ respectively, confirming that memory is exclusively BSGS’s bottleneck and not a concern for the other three.

5.4 Pohlig-Hellman Vulnerability

For smooth primes, Pohlig-Hellman solves 40+ bit DLPs in milliseconds because $p - 1$ decomposes into subgroups of size at most 53, each solved by brute force in negligible time. On safe primes, the largest factor of $p - 1$ approaches $p/2$, so the subgroup DLP is nearly as

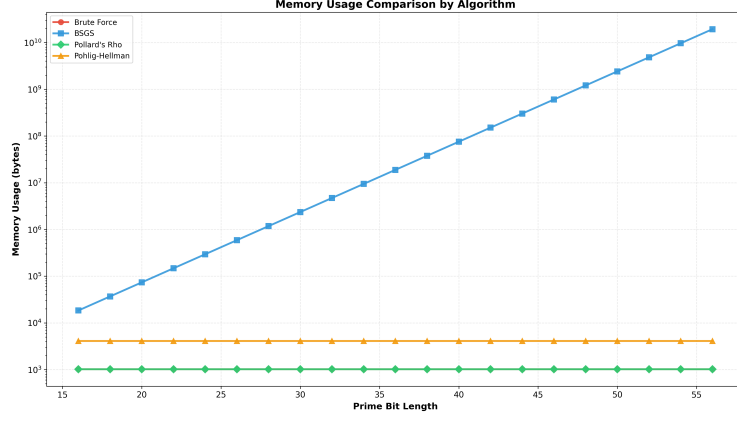


Figure 3: Memory consumption comparison across algorithms and bit-lengths.

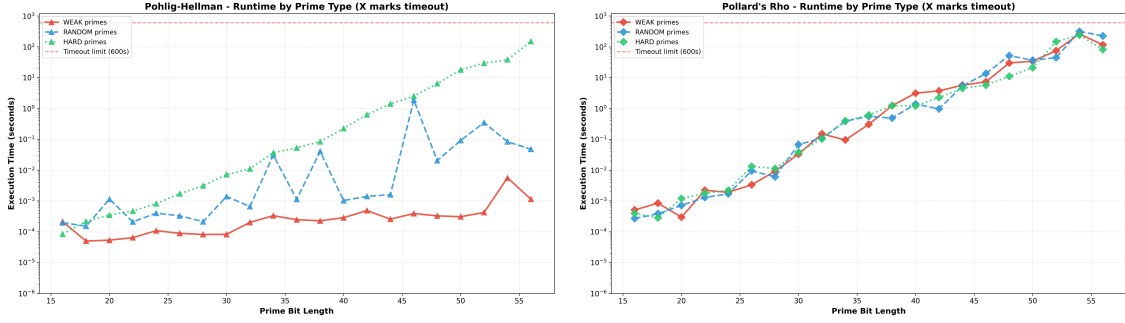


Figure 4: Pohlig-Hellman (left) and Pollard's ρ (right) performance across all three datasets.

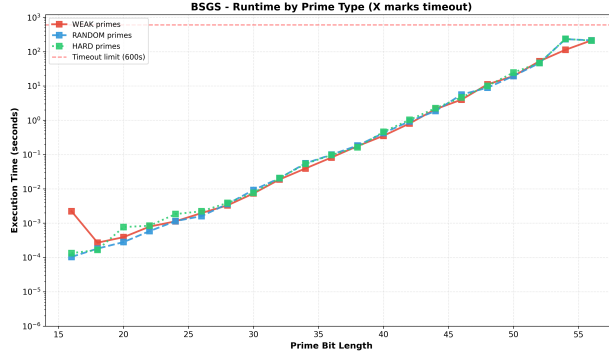


Figure 5: BSGS performance across all three datasets.

hard as the original problem and performance degrades to that of the underlying brute-force subgroup solver plus CRT overhead.

5.5 Strengths and Weaknesses

- **Brute Force:** *Strength* — trivially simple, $O(1)$ memory, always correct. *Weakness* — $O(2^n)$ time makes it useless beyond ≈ 28 bits.

- **BSGS:** *Strength* — deterministic $O(2^{n/2})$ time, guaranteed to find the answer. *Weakness* — $O(2^{n/2})$ memory causes it to run out of RAM before time becomes the limiting factor at large bit-lengths.
- **Pollard’s ρ :** *Strength* — $O(2^{n/2})$ expected time with $O(1)$ memory, making it the most practical general-purpose algorithm. *Weakness* — probabilistic; can occasionally require multiple restarts and has no hard runtime guarantee.
- **Pohlig-Hellman:** *Strength* — solves DLP on smooth primes almost instantly regardless of bit-length. *Weakness* — provides no advantage on safe primes; performance collapses to that of a generic solver.

6 Conclusion

The empirical results confirmed theoretical predictions across all four algorithms. Brute Force illustrates why the DLP is hard in general; BSGS shows the ceiling imposed by memory on time-memory trade-offs; Pollard’s ρ demonstrates how to match BSGS’s time complexity while sidestepping the memory cost; and Pohlig-Hellman shows that structural properties of $p - 1$ can completely determine hardness independent of bit-length.

The most striking result is the $19,460\times$ runtime difference for Pohlig-Hellman between a smooth and a safe prime of the same size. This is not a marginal effect — it is the reason modern standards mandate safe primes. A 48-bit smooth prime is solved in under 1 ms; the same bit-length with a safe prime takes over 6 seconds with the same algorithm. Scaled to the 2048-bit primes used in real protocols, the structural choice is the difference between a system that is broken in microseconds and one that resists all known attacks.

Future work: A natural extension is to implement and benchmark the Elliptic Curve Discrete Logarithm Problem (ECDLP), which resists index calculus methods, or to explore sub-exponential methods like the Number Field Sieve (NFS) for the standard DLP.

References

- [1] Diffie, W., & Hellman, M. (1976). *New directions in cryptography*. IEEE Transactions on Information Theory, 22(6), 644–654.
- [2] Kivinen, T., & Kojo, M. (2003). *More Modular Exponential (MODP) Diffie-Hellman groups for Internet Key Exchange (IKE)*. RFC 3526, Internet Engineering Task Force.
- [3] National Institute of Standards and Technology. (2023). *Digital Signature Standard (DSS)*. FIPS PUB 186-5.
- [4] Shanks, D. (1971). *Class number, a theory of factorization, and genera*. Proc. Symp. Pure Math. 20, 415–440.
- [5] Pollard, J. M. (1978). *Monte Carlo methods for index computation (mod p)*. Mathematics of Computation, 32(143), 918–924.

- [6] Pohlig, S., & Hellman, M. (1978). *An improved algorithm for computing logarithms over $GF(p)$ and its cryptographic significance*. IEEE Transactions on Information Theory, 24(1), 106–110.